

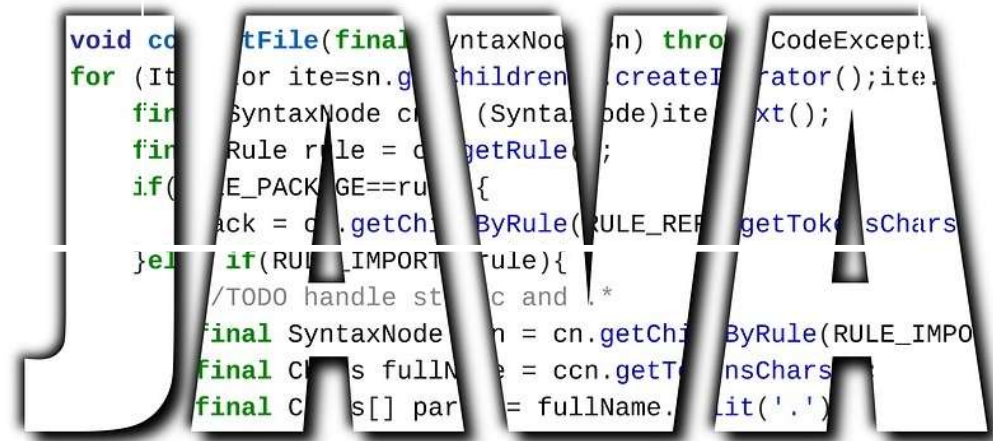
Module 61CSE215: Object-oriented Programming with Java

Java Overview

Trần Hồng Ngọc, Ph.D.
ngoc.th@vgu.edu.vn

Content

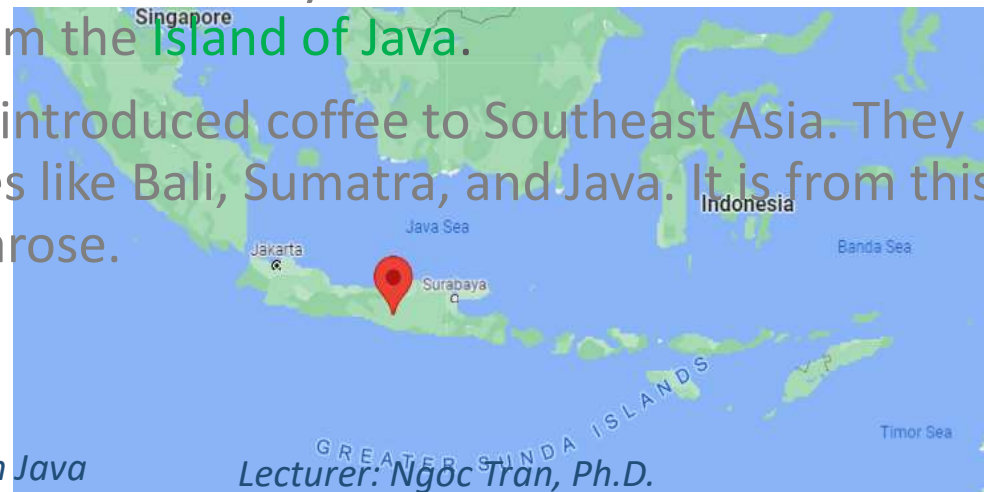
- What is Java?
- History
- Applications of Java
- Java based Tools
- Features of Java
- JDK, JRE and JVM
- Java Versions
- A Simple Java Code Sample
- A way Java Works
- Anatomy of a Java Program



What is Java?

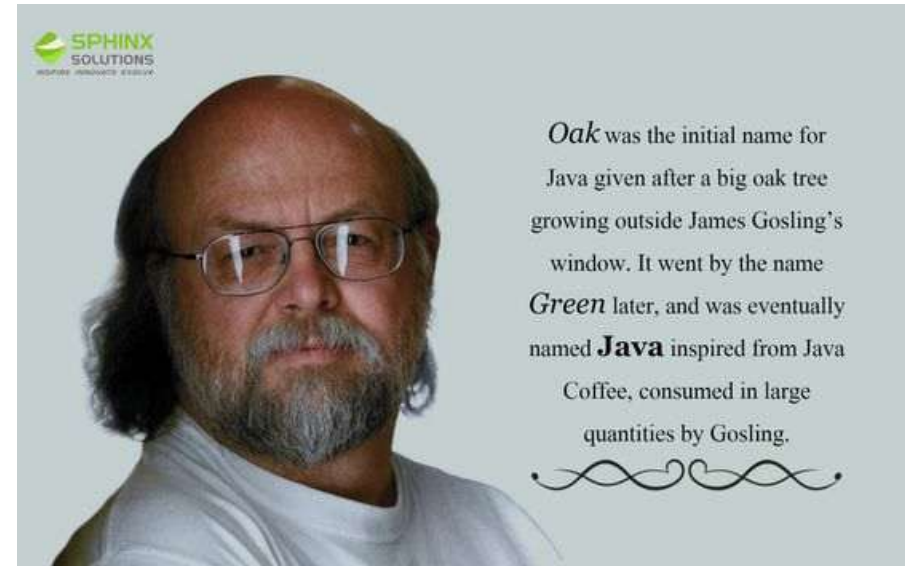


- Is one of the most popular software development languages nowadays.
- Is a high-level language, an object oriented programming language.
- Coffee names are categorized based on 1) How coffee is made (e.g., espresso, drip), 2) A specific beverage made with coffee (e.g., mocha, cappuccino), 3) Coffee's origins and history. "Java" falls into this third category → "Java" comes from the **Island of Java**.
- During the 1600s, the Dutch introduced coffee to Southeast Asia. They brought coffee trees to places like Bali, Sumatra, and Java. It is from this island that the name "java" arose.



History

- 1991
 - Firstly, Java was called **Greentalk** by **James Gosling**, and the file extension was .gt.
 - After that, it was called **Oak** and was developed as a part of the Green project.



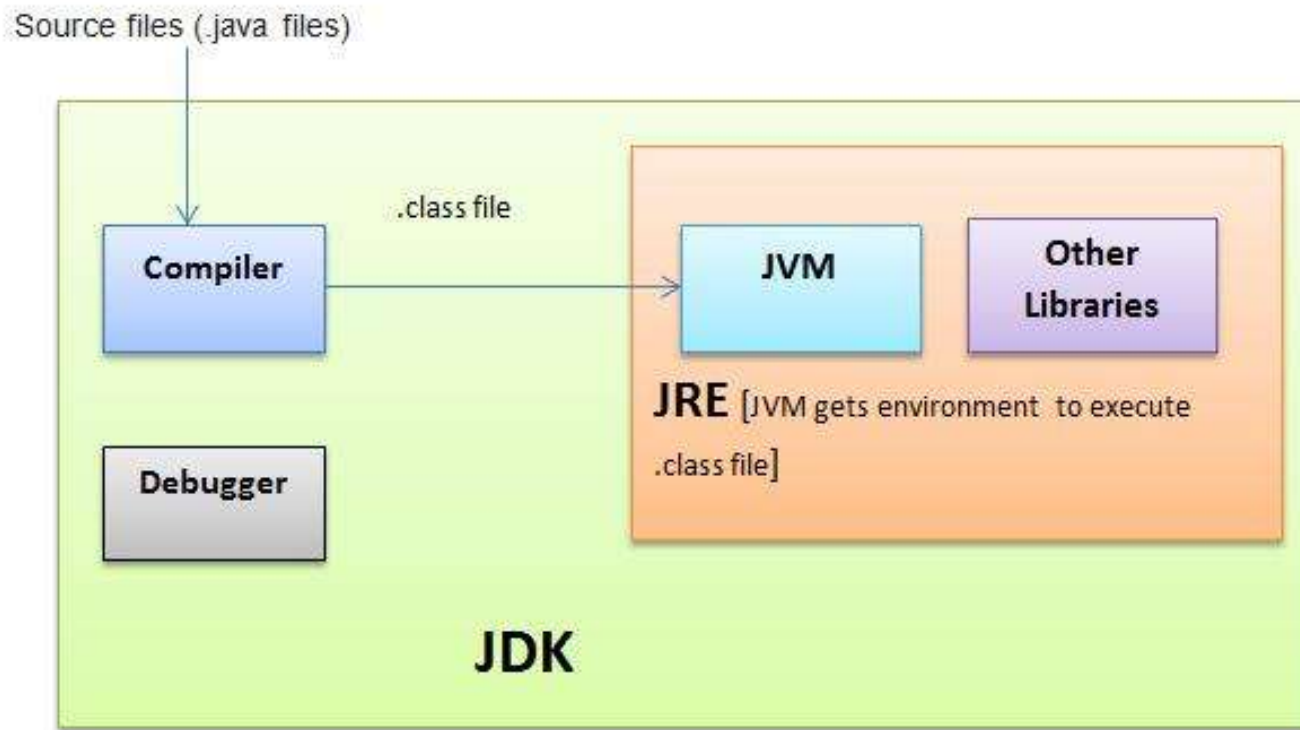
James Gosling, the creator of Java
Birthdate: 19-May-1955 in Calgary, Alberta, Canada

- Gosling designed Java with a C/C++-style syntax that system and application programmers would find familiar.
- Originally, it was developed for handling portable devices and set-top boxes. Oak was a massive failure.

History

- 1995
 - Sun changed the name to **Java**.
 - Sun modified the language to take advantage of the burgeoning www (World Wide Web) development business.
 - It promised "Write Once, Run Anywhere" (WORA), providing no-cost run-times on popular platforms.
- 2009
 - Oracle Corporation acquired Sun Microsystems and took ownership of three key Sun software assets: Java, MySQL, and Solaris.

Java Programming Enviroment



Java Development Kit (JDK)

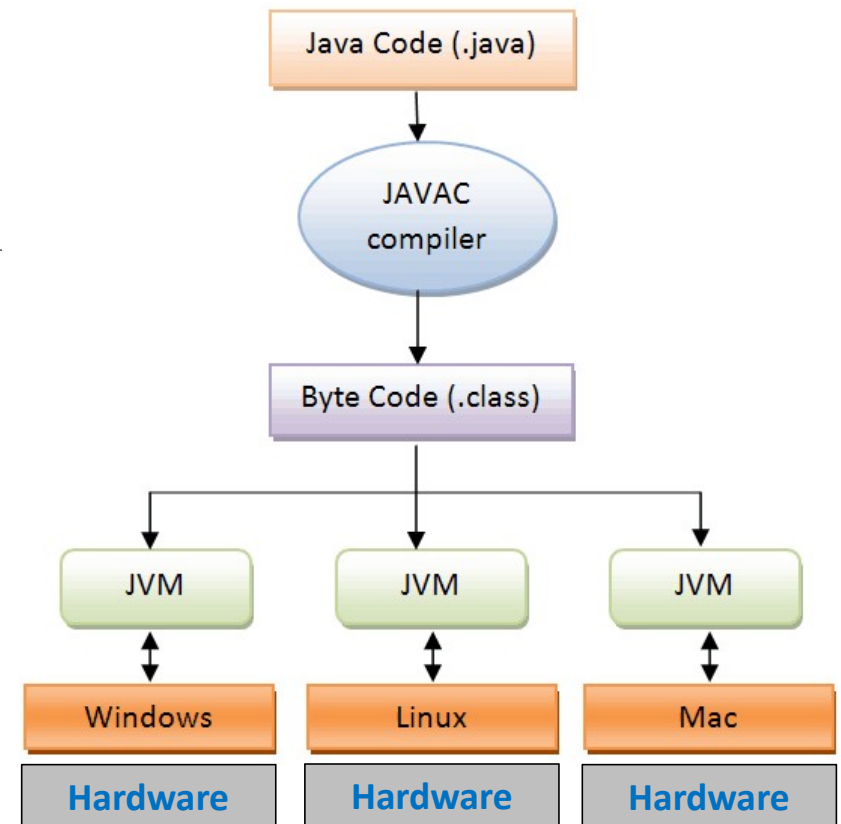
- **Java Development Kit (JDK)** is a **superset** of a JRE and contains tools for Java programmers, e.g. javac compiler.
- **JDK Editions:**
 - **Java Standard Edition (J2SE)**
 - J2SE can be used to develop client-side standalone applications or applets.
 - **Java Enterprise Edition (J2EE)**
 - J2EE can be used to develop server-side applications such as Java servlets and Java ServerPages.
 - **Java Micro Edition (J2ME).**
 - J2ME can be used to develop applications for mobile devices such as cell phones.
- **Download:** at Oracle website

Jave Runtime Environment (JRE)

- **Java Runtime Environment (JRE)** is a software package that contains what is required to run a Java program.
- It includes a Java Virtual Machine implementation together with an implementation of the Java Class Library.
- Download: at Oracle website

Java Virtual Machine (JVM)

- **Java Virtual Machine (JVM)** is a virtual machine that enables a computer to run Java programs as well as programs written in other languages that are also compiled to Java bytecode.
- **JVM**
 - Loads code
 - Verifies code
 - Executes code
 - Provides runtime environment



Java Versions

	Version	Date
1	JDK Alpha & Beta	1995
2	JDK1.0	January 1996
3	JDK 1.1	February 1997
4	J2SE 1.2	December 1998
5	J2SE 1.3	May 2000
6	J2SE 1.4	February 2002
7	J2SE 5.0	September 2004
8	Java SE 6	December 2006
9	Java SE 7	July 2011
10	Java SE 8	March 2014
11	Java SE 9	September 2017
12	Java SE 10	March 2018
13	Java SE 11	September 2018

	Version	Date
14	Java SE 12	March 2019
15	Java SE 13	September 2019
16	Java SE 14	March 2020
17	Java SE 15	September 2020
18	Java SE 16	March 2021
19	Java SE 17	September 2021
20	Java SE 18	March 2022
21	Java SE 19	September 2022
22	Java SE 20	March 2023
23	Java SE 21	September 2023
24	Java SE 22	March 2024
25	Java SE 23	September 2024
JDK 22 will receive updates under these terms, until September 2024, when it will be superseded by JDK 23.		

11

Intergrated Developing Environment (IDE)

1. NetBeans
2. Eclipse: <https://www.eclipse.org/downloads/>
3. IntelliJ IDEA
4. Jdeveloper
5. JDoodle: <https://www.jdoodle.com/online-java-compiler/> for quickly running a small segment of Java code.
6. VSCode

Java is an Object-Oriented Programming Language

- What is Object?
- What is an Object-Oriented Programming (OOP)?

Object

- An **entity** that has state and behavior is known as an object
- It can be **physical** (tangible) or **logical** (intangible).
 - Examples of tangible objects are person, chair, bike, marker, table.
 - An example of an intangible object is the banking system, human resource management system.

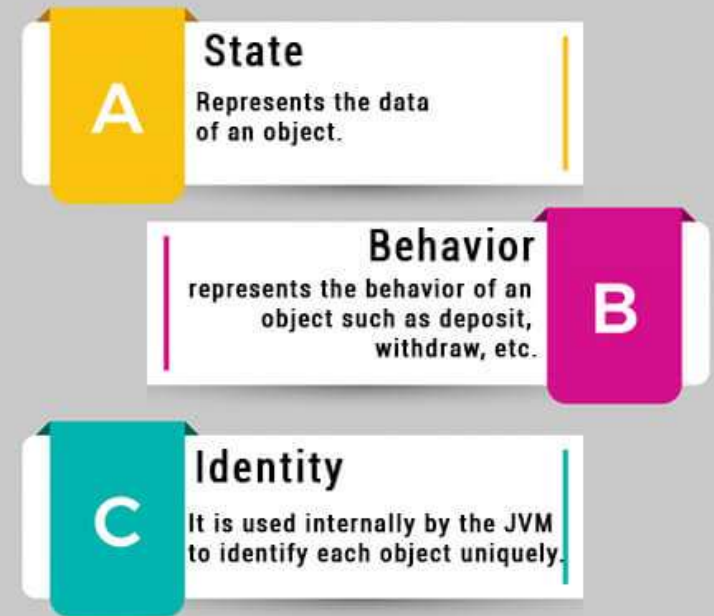
Object

- An object has three characteristics: **state**, **behavior** and **identity**.
- E.g., dog is an object.



- **States:** Dog has **name** Pie, her **breed** is Corgi, her **size** is small, her **color** is red and white and she is 1 year old.
- **Behaviors:** eat, sleep, sit, run.

Characteristics of Object



Source: Javatpoint.com

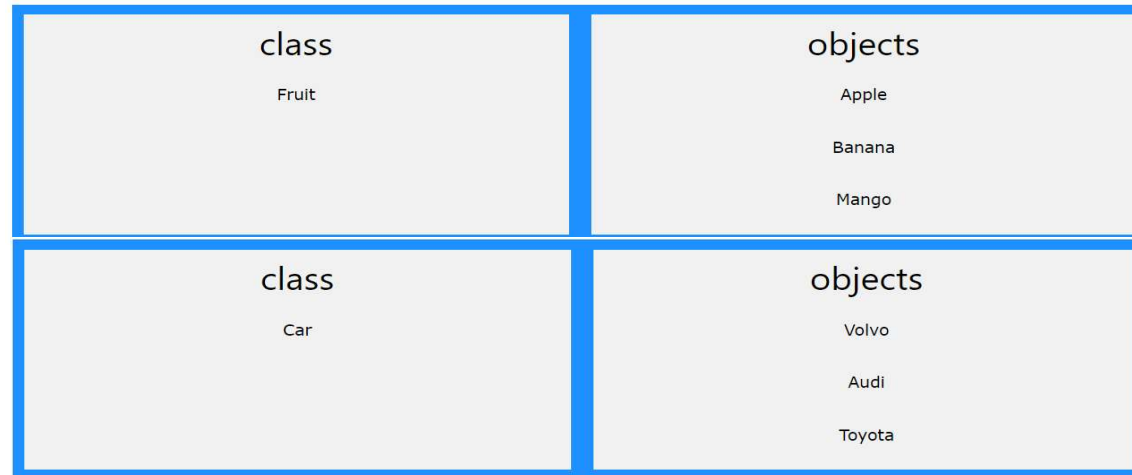
Class

- A **class** is a template or blueprint from which objects are created.
- **Class** is a basic concept of OOP which revolve around the real-life entities.
- **Class** in Java determines how an object will behave and what the object will contain.
- Syntax

```
class <class_name>{  
    fields;           //states  
    methods;         //bahaviors  
}
```

Class vs Object

- A class is a template for objects, and an object is an instance of a class.
- When the individual objects are created, they inherit all the variables and methods from the class.



From one class, many instances of class so-called objects are created.

Java is an object oriented programming language

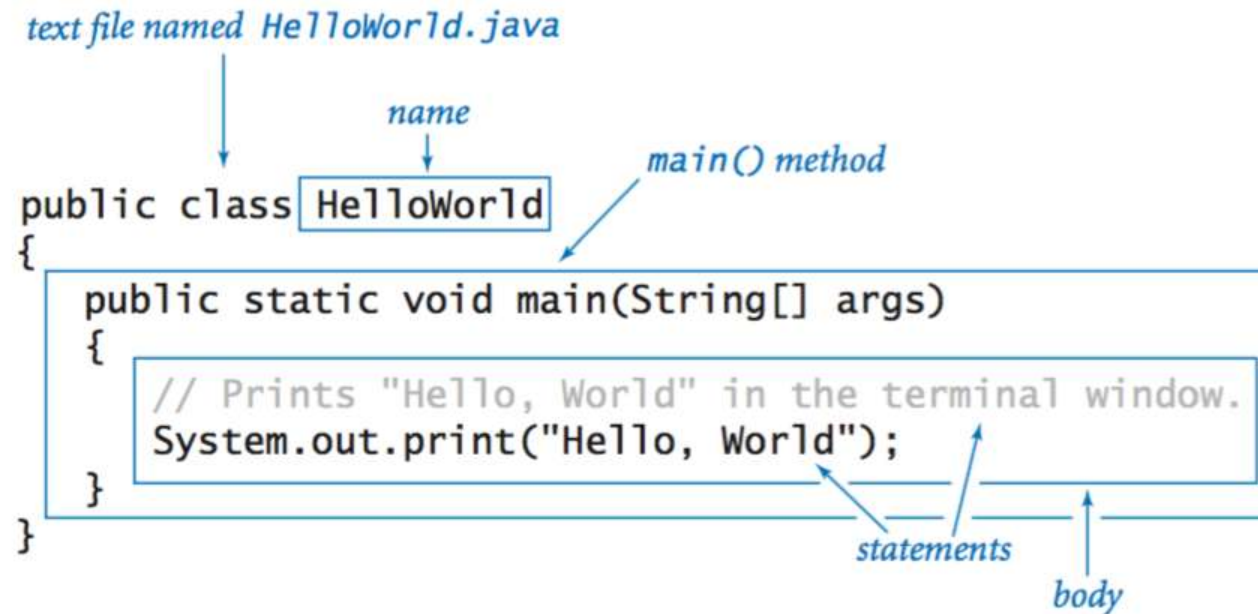
Class vs Object

- **Object** is an instance of a class.
- Syntax to declare an object.

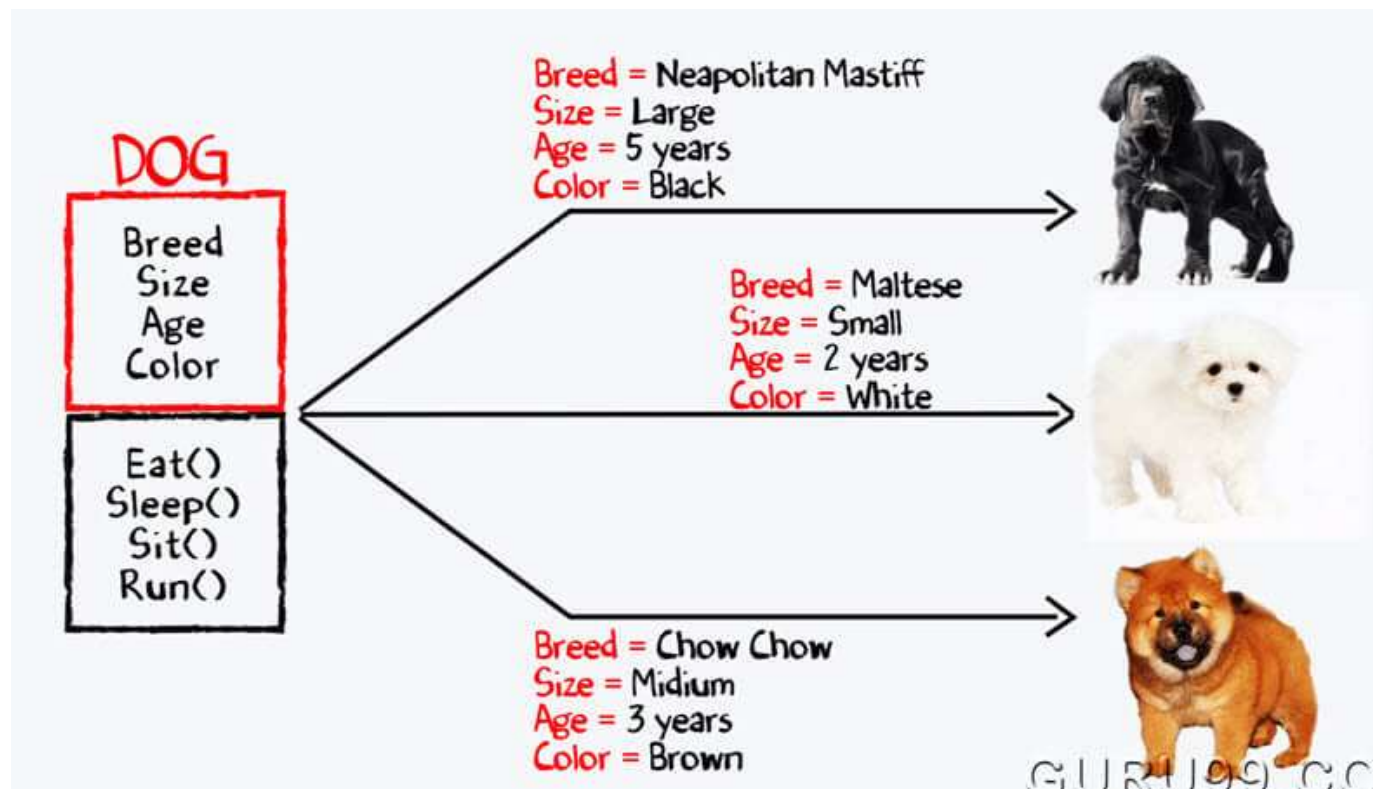
ClassName *ReferenceVariable* = *new* *ClassName*();

- To create an object:
 - Defining a **class** describing the common features (states) of all objects in the same classification of the requested object.
 - Declaring the **object** with the defined class.

First Hello



Example



Example – Solution 1

```
34 // Class Declaration
35 public class Dog {
36     // Instance Variables
37     String breed;
38     String size;
39     int age;
40     String color;
41
42
43     // method 1
44     public String getInfo() {
45         return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
46     }
47
48
49     public static void main(String[] args) {
50         Dog maltese = new Dog();
51         maltese.breed="Maltese";
52         maltese.size="Small";
53         maltese.age=2;
54         maltese.color="white";
55         System.out.println(maltese.getInfo());
56     }
57 }
```

Example – Solution 2

```
60 // Class Declaration
61 class Dog {
62     // Instance Variables
63     String breed;
64     String size;
65     int age;
66     String color;
67
68
69     // method 1
70     public String getInfo() {
71         return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
72     }
73 }
74 public class Execute{
75     public static void main(String[] args) {
76         Dog maltese = new Dog();
77         maltese.breed="Maltese";
78         maltese.size="Small";
79         maltese.age=2;
80         maltese.color="white";
81         System.out.println(maltese.getInfo());
82     }
83 }
```

Exercise

Declare 3 other objects of Neapolitan Mastiff, Chow Chow, and Corgi.

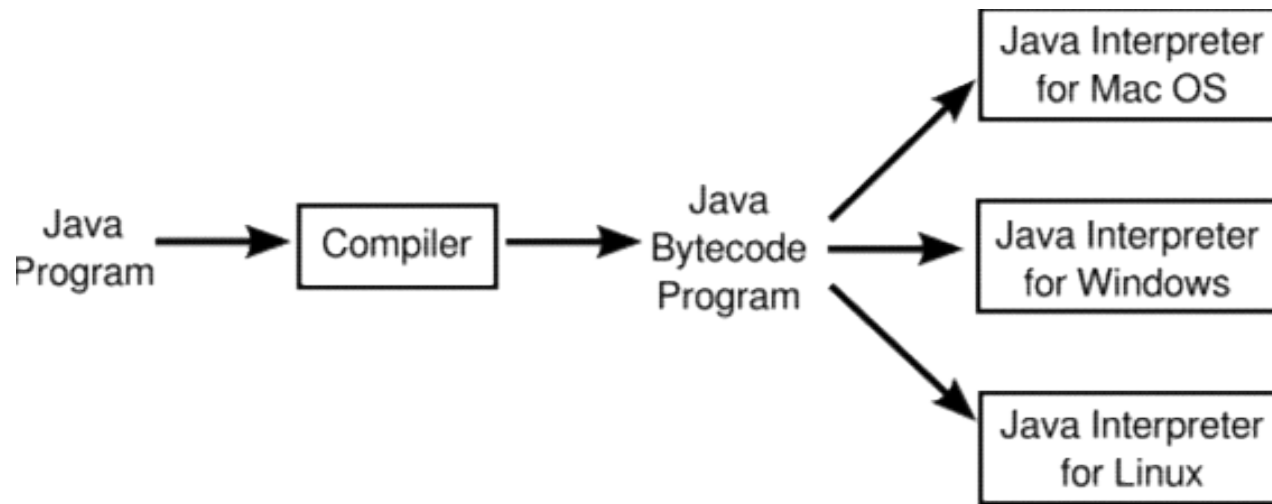
C is a Procedural Programming Language

- Procedural programming is about writing **functions** that perform operations on the data.
- **The program gets larger in time when more functions are created.**
 - It is harder to manage and maintain the code in a huge pile of functions.
 - Reusing the code is limited (we can reuse the common functions but we cannot modify them when reusing them).

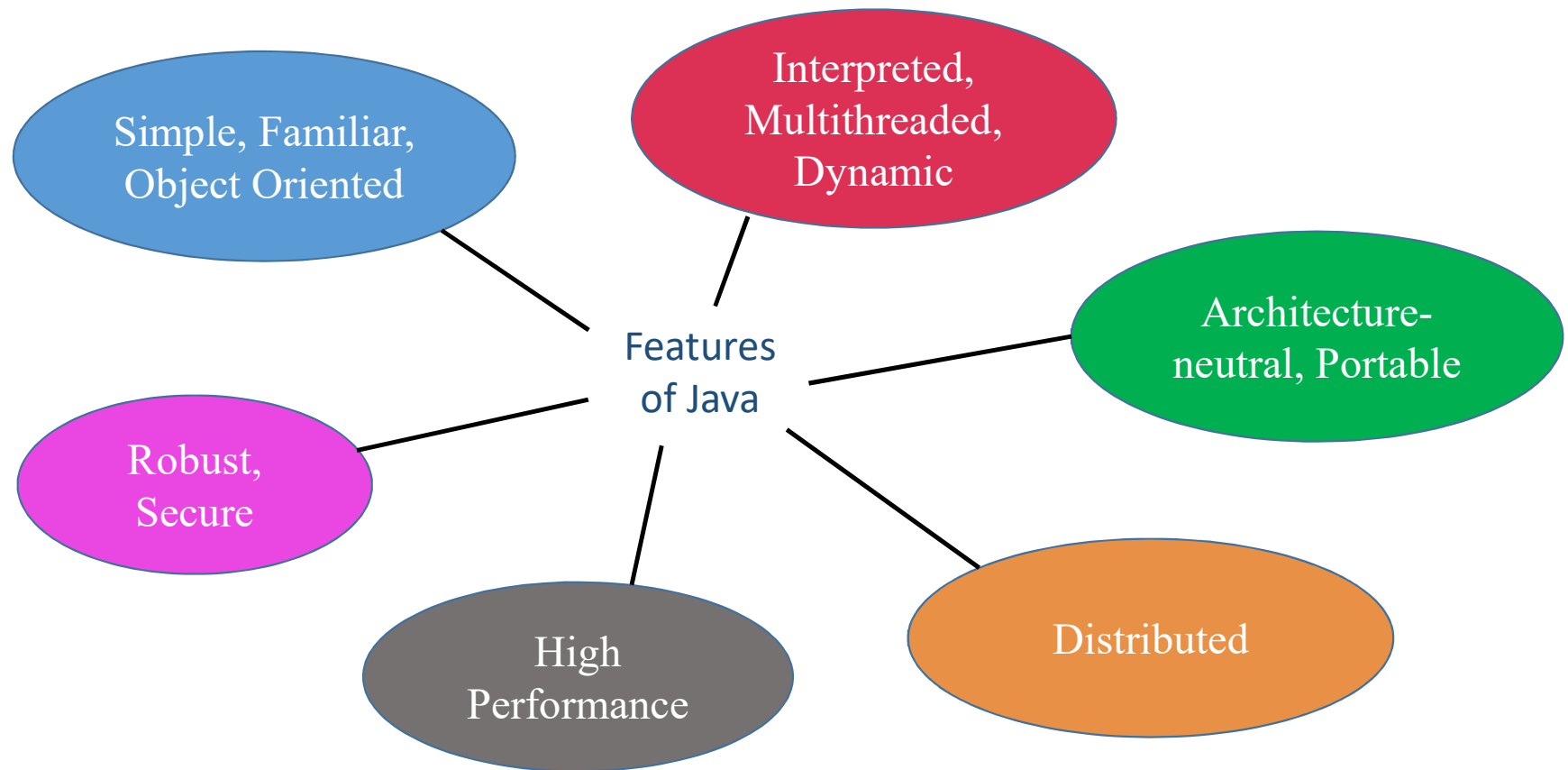
Java is an Object-Oriented Programming Language

- OOP is at the core of Java.
- OOP is about creating objects that contain both **data** (or **states**) and **methods** (or **behaviors**).
- OOP has several advantages over procedural programming:
 - OOP provides a clear structure for the programs.
 - OOP helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug.
 - OOP makes it possible to create full reusable applications with less code and shorter development time.

Java Interpreter



Features of Java



Applications of Java

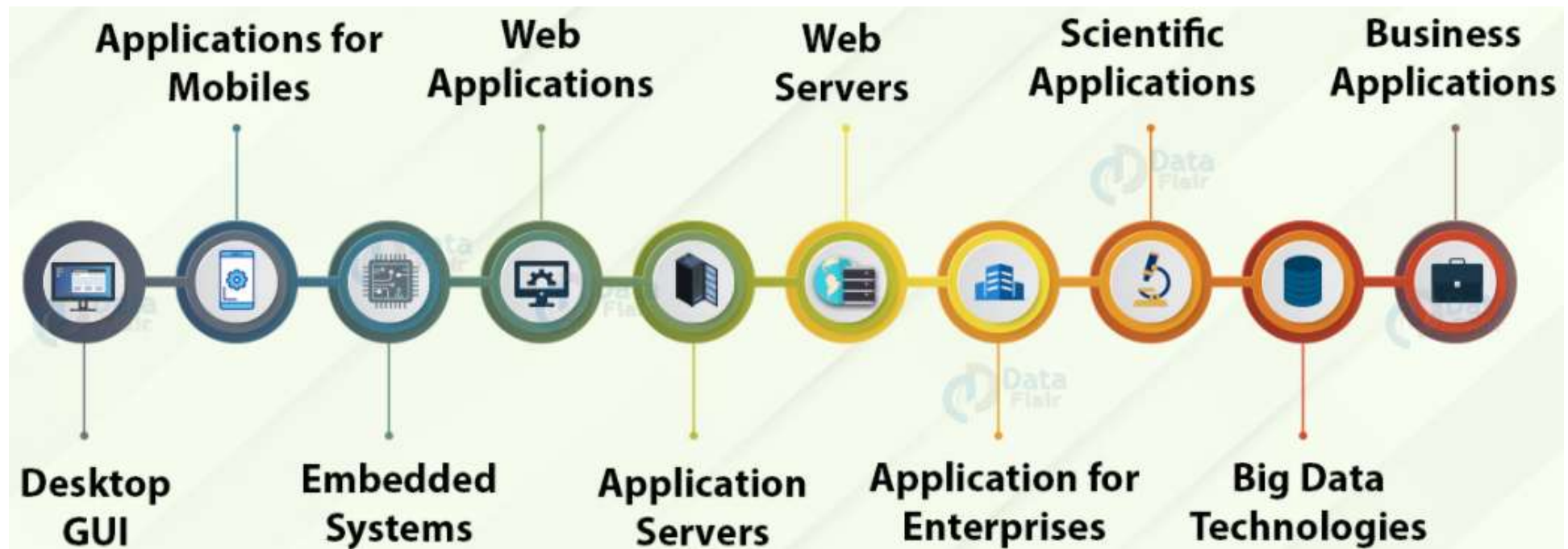
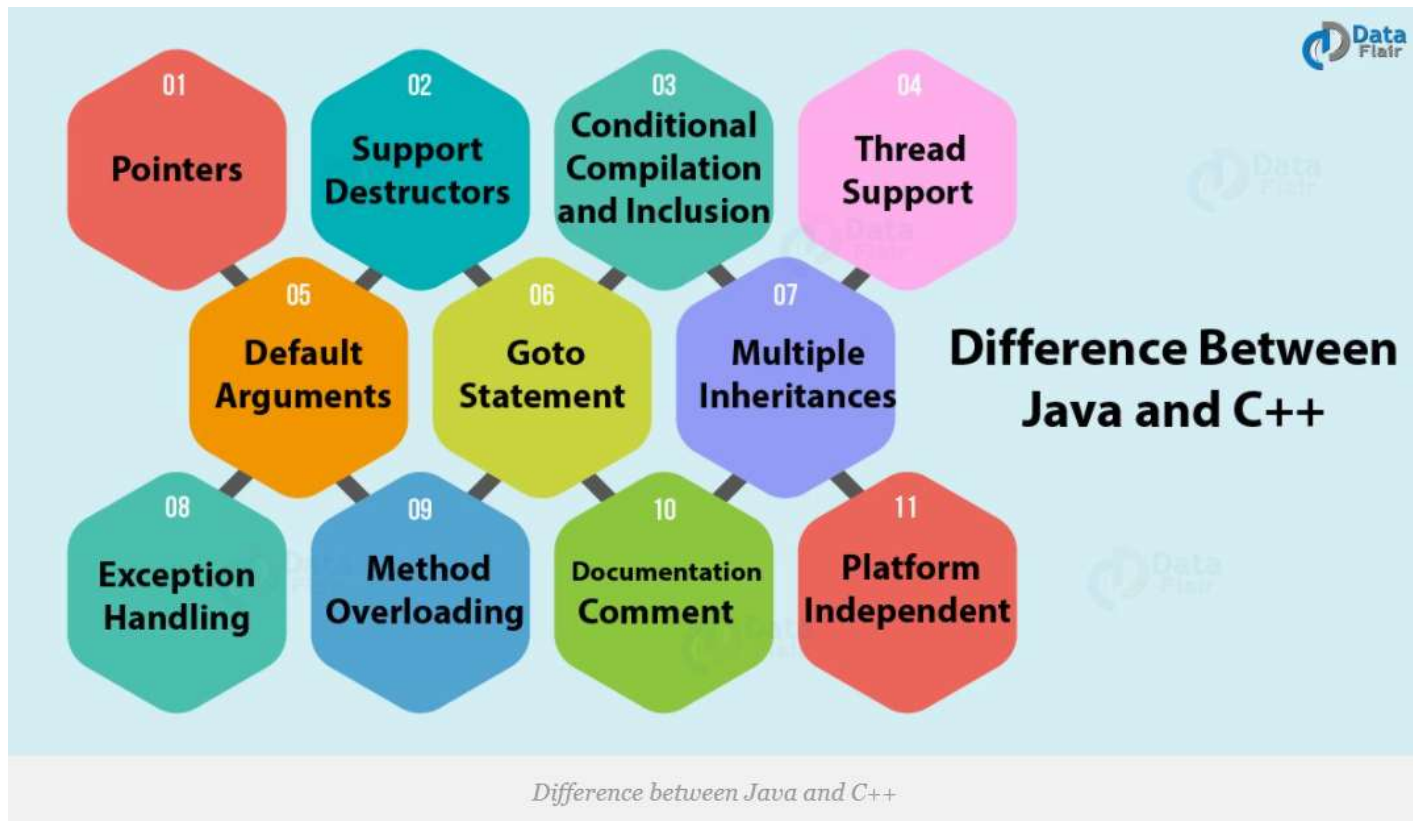


Image Source: DataFlair

Some Tools

- Desktop GUI: Abstract Windowing Toolkit, JavaFX and Swing.
- Applications for Mobiles are built using Android Studio, Flutter, Kotlin.
- Web Applications are built using Servlets, Struts and JSPs.
- Web Servers in Java as Tomcat, Apache, JBoss, GlassFish.

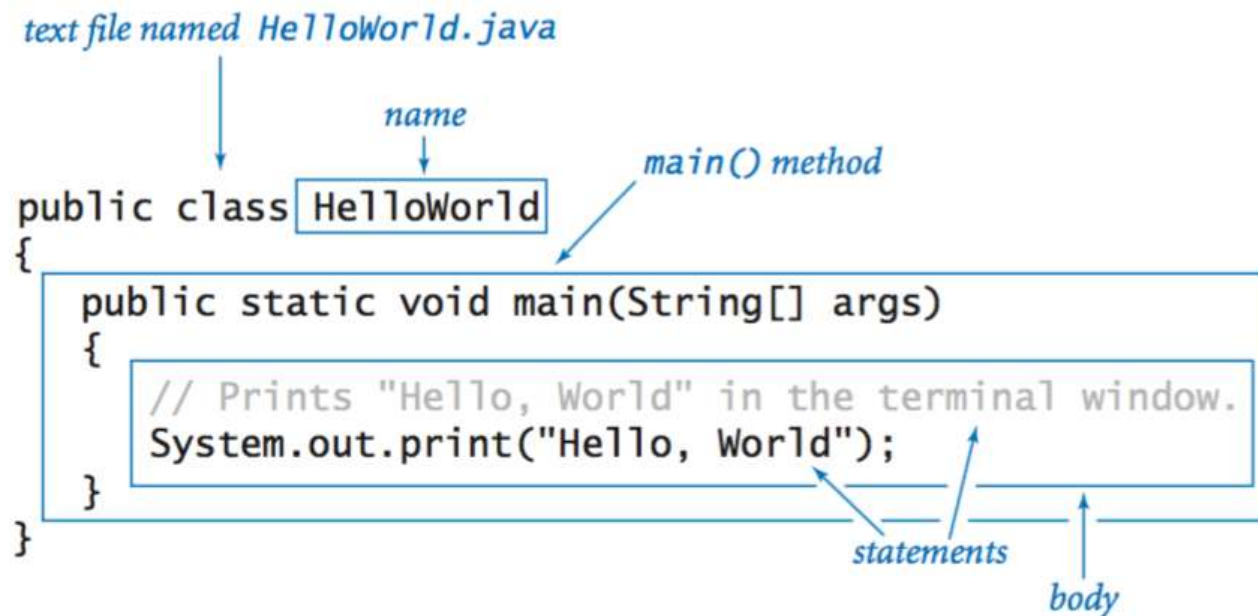
With the other OOP languages: Java vs C++



Exercise: Software Installation Preparation

- Install JDK
 - Installing this software means JVM and JRE are installed as well as JDK contains them.
- Install Java IDE

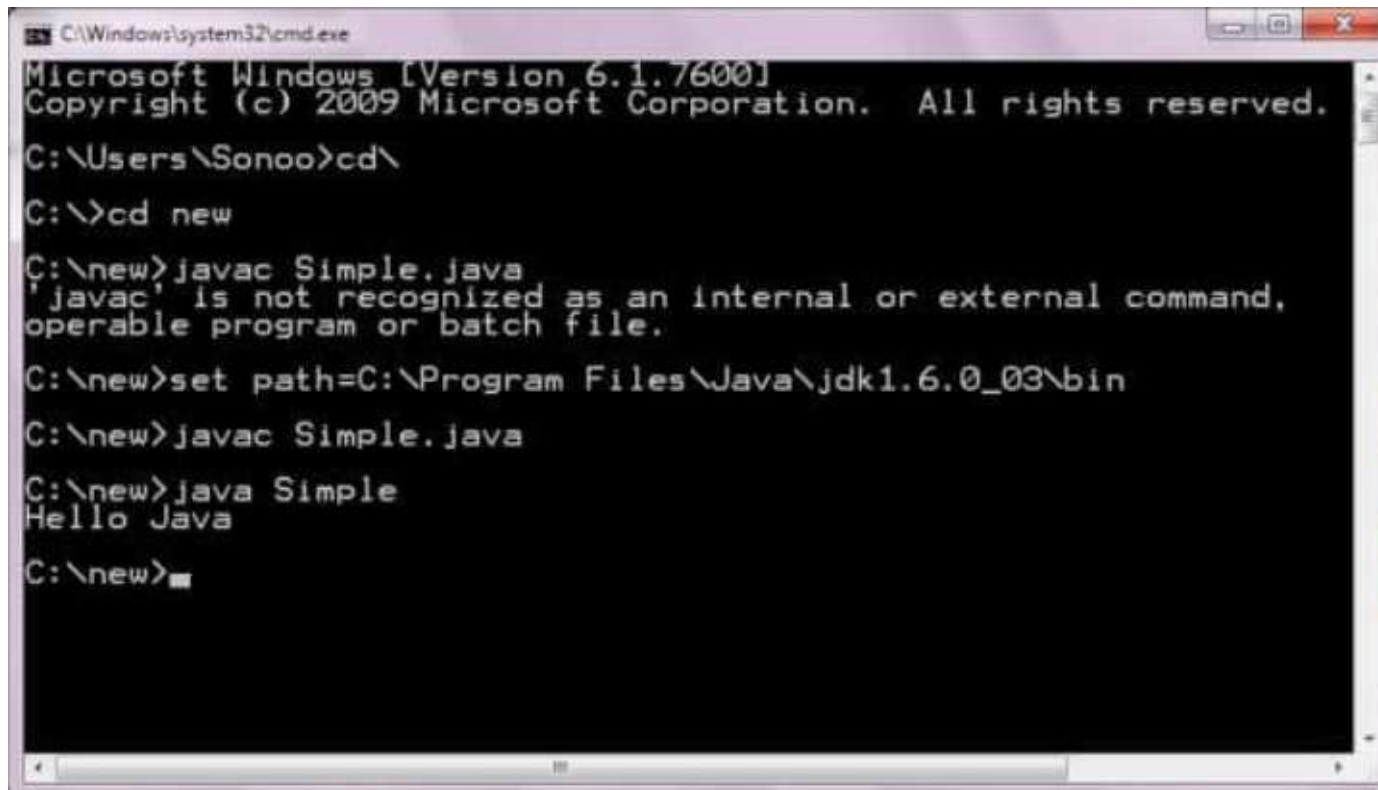
A simple Java code sample



Compile & Run on Console

1. Run the code in IDE.
2. Run the code in the terminal (Linux/Ubuntu), or on the console (press Windows Key + R then type “cmd”).
 - To compile: `javac -d directory javafilename`
 - E.g., `javac -d . Helloworld.java`
 - To run: `java javafilename`
 - E.g., `java Helloworld`

Compile & Run on Console



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Sonoo>cd\
C:\>cd new
C:\new>javac Simple.java
'javac' is not recognized as an internal or external command,
operable program or batch file.

C:\new>set path=C:\Program Files\Java\jdk1.6.0_03\bin
C:\new>javac Simple.java
C:\new>java Simple
Hello Java
C:\new>
```

Anatomy of a Java program

1. Block
2. Statement
3. Comment
4. Class
5. Package
6. Reserved word
7. Modifier
8. Method
9. The main method

1. Block

- A pair of braces ‘{’, ‘}’ in a program forms a **block** that groups components of a program.

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

Diagram illustrating blocks in the code:

- The entire code is enclosed in a **Class block** (indicated by a long arrow from the opening curly brace to the closing curly brace).
- The `main` method is enclosed in a **Method block** (indicated by a shorter arrow from its opening curly brace to its closing curly brace).

```
1 class BlockDemo {
2     public static void main(String[] args) {
3         boolean condition = true;
4         if (condition) { // begin block 1
5             System.out.println("Condition is true.");
6         } // end block 1
7         else { // begin block 2
8             System.out.println("Condition is false.");
9         } // end block 2
10    }
11 }
```

2. Statement

- A **statement** represents **an action** or **a sequence of actions**.
- Every statement in Java ends with a semicolon (;).

```
1  int cadence = 0;
2  anArray[0] = 100;
3  System.out.println("Element 1 at index 0: " + anArray[0]);
4
5  int result = 1 + 2; // result is now 3
6  if (value1 == value2)
7      System.out.println("value1 == value2");
```

```
22  // assignment statement
23  aValue = 8933.234;
24  // increment statement
25  aValue++;
26  // method invocation statement
27  System.out.println("Hello World!");
28  // object creation statement
29  Bicycle myBike = new Bicycle();
30  // declaration statement
31  double aValue = 8933.234;
```

3. Comment

- Comments
 - **Single-line statement**: the line is preceded by two slashes (//)
 - When the compiler sees //, it ignores all text after // in the same line.
 - **Multi-line statement**: enclosed between /* and */ in one or multiple lines.
 - When it sees /*, it scans for the next */ and ignores any text between /* and */.

```
// This is a comment  
System.out.println("Hello World");
```

```
System.out.println("Hello World"); // This is a comment
```

```
/* The code below will print the words Hello World  
to the screen, and it is amazing */  
System.out.println("Hello World");
```

4. Class

- The *class* is the essential Java construct.
- A class is a template or blueprint for objects.
- To program in Java, you must understand classes and be able to write and use them.
- A program is defined by using one or more classes.

5. Package

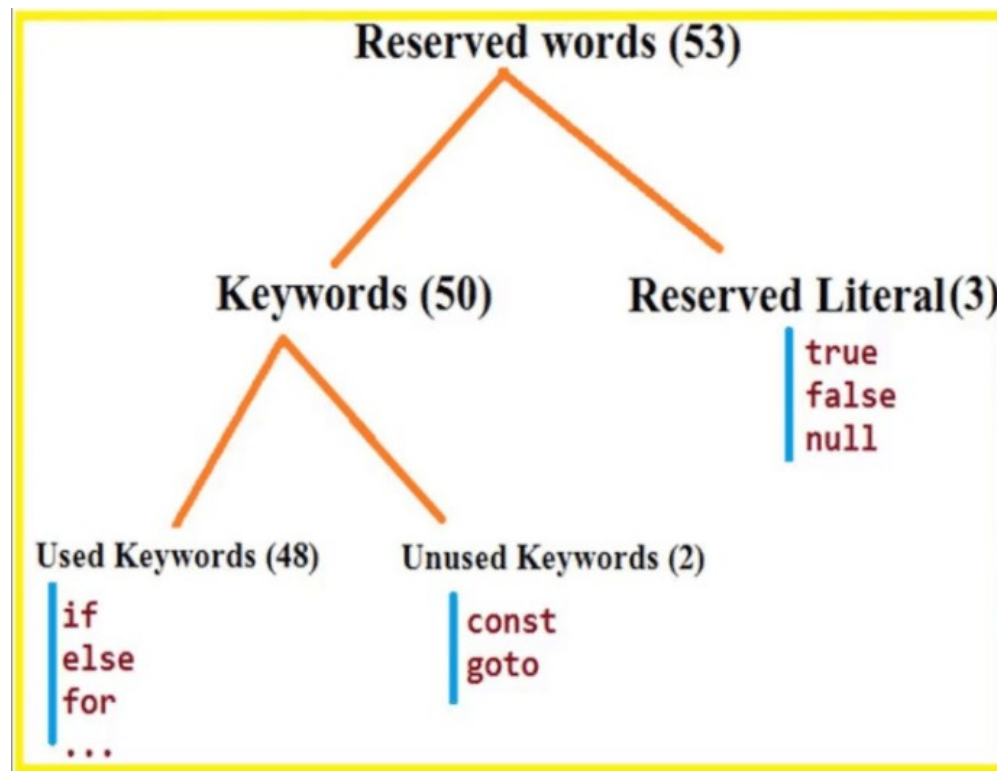
- The **package** keyword is used to create a package in java.
- The second line in the program (**package mypack;**) specifies a package name, **mypack**, for the class **Simple**.
- Java IDE compiles the source code in **Simple.java**, generates **Simple.class**, and stores **Simple.class** in the **mypack** folder.

```
//save as Simple.java
package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

6. Reserved Words

- **Reserved words** or **keywords** are words that have a specific meaning to the compiler and cannot be used for other purposes in the program.
- E.g., when the compiler sees the word **class**, it understands that the word after **class** is **the name for the class**. Other reserved words are **public**, **static**, and **void**, etc.

6. Reserved Words



6. Reserved Words

abstract	else	interface	super
boolean	extends	long	switch
break	false ^a	native	synchronized
byte	final	new	this
case	finally	null ^a	throw
catch	float	package	throws
char	for	private	transient
class	goto ^b	protected	true ^a
const ^b	if	public	
continue	implements	return	
default	import	short	
do	instanceof	static	
double	int	strictfp ^c	

restricts floating-point calculations to ensure portability. The `strictfp` command was introduced into Java with the JVM version 1.2 and is available for use on all currently updated Java VMs

Note. ^a true, false, and null are reserved words.
^b indicates a keyword that is not currently used. ^c indicates a keyword that was added for Java 2

6. Reserved Words

- In Java, the **strictfp** keyword is used to force the precision of floating point calculations (float or double) in Java conform to **IEEE's 754 standard**, explicitly.
- Without using **strictfp** keyword, the floating point precision depends on target platform's hardware, i.e. CPU's floating point processing capability. In other words, using **strictfp** ensures result of floating point computations is always same on all platforms.

abstract	else	interface	super
			switch
			synchronized
			this
			throw
			throws
			transient
		ed	true ^a
continue	implements	return	
default	import	short	
do	instanceof	static	
double	int	strictfp	

restricts floating-point calculations to ensure portability. The **strictfp** command was introduced into Java with the JVM version 1.2 and is available for use on all currently updated Java VMs

Note. ^a true, false, and null are reserved words.

^b indicates a keyword that is not currently used. ^c indicates a keyword that was added for Java 2

6. Reserved Words

- In Java, the **strictfp** keyword is used to force the precision of floating point calculations (float or double) in Java conform to **IEEE's 754 standard**, explicitly.
- Without using **strictfp** keyword, the floating point precision depends on the CPU's floating point hardware. Using **strictfp** keyword, the floating point computations is always the same.

```

1  strictfp class StrictFPClass {
2      double num1 = 10e+102;
3      double num2 = 6e+08;
4      double calculate() {
5          return num1 + num2;
6      }
7  }
    
```

floating-point calculations to ensure accuracy. The **strictfp** command was introduced in Java with the JVM version 1.2 and is required for use on all currently updated Java

abstract	else	interface	super
			switch
			synchronized
			this
			throw
			rows
			transient
			no a
do	instanceof	static	VMs
double	int	strictfp	

Note. ^a true, false, and null are reserved words. ^b indicates a keyword that is not currently used. ^c indicates a keyword that was added for Java 2

7. Modifiers

- Java uses certain **reserved words** called **modifiers** that **specify** the **properties** of the data, methods, and classes and **how** they can be used.
- E.g., **public**, **static**, **private**, **final**, **abstract**, and **protected**.
 - A **public** data/method/class can be accessed by other programs.
 - A **private** data/method cannot be accessed by other programs.

Access Modifiers	Non-Access Modifiers
private default or No Modifier protected public	static final abstract synchronized transient volatile strictfp

8. Method

- Describes a **behavior** of a class.
- E.g., System.out.println("Welcome to Java!")
 - It is **a method**: a collection of statements that **performs a sequence of operations to display a message on the console**.
 - It is used by invoking a statement with a **string** argument enclosed within parentheses.
 - In this case, the argument is "Welcome to Java!"
 - Use a different argument to print a different message.
- It can be used even **without** fully understanding the details of how it works.

9. Main Method

- The main method provides the control of program flow. The Java application is executed by invoking the main method.
- In a Java project, **there may be more than one class**. However, there is **only one class can possess the main method**.

The main method looks like this:

```
76 public static void main(String[] args) {  
77     //statements  
78 }
```

Object-Oriented Programming

1. Two Paradigms
2. Abstraction
3. The Three OOP Principles
 - a) Encapsulation
 - b) Inheritance
 - c) Polymorphism

Two Paradigms

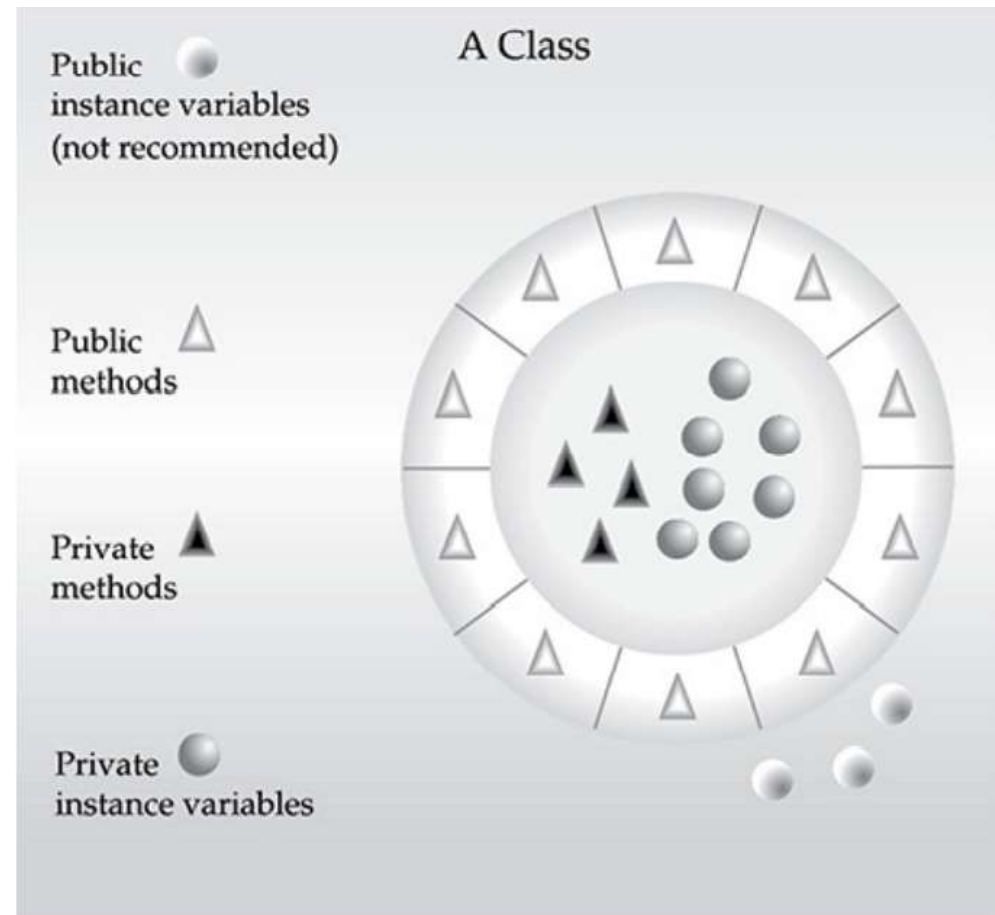
- All computer programs consist of two elements: **code** and **data**.
- The two paradigms govern **how a program is constructed**.
 1. **Process-oriented model** can be thought of as code acting on data.
 - Problems with this approach appear as programs grow larger and more complex
 - E.g., Procedural languages such as C employ this model (E.g. on slide 24).
 2. **Object-oriented programming** organizes a program around its data (that is, **objects**) and a set of well-defined **interfaces** to that data.
 - An object-oriented program can be characterized as data controlling access to code.

Abstraction

- Is one of the **most important** element of object-oriented programming.
- Is the concept of object-oriented programming that **shows only essential attributes** and **hides unnecessary internal implementation details**.
- It helps in **reducing programming complexity and efforts**.
- **Example**
 - The coffee machine in your kitchen.
 - You just need to know which methods of the object are available to call and which input parameters are needed to trigger a specific operation.
 - But you don't need to understand how this method is implemented and which kinds of actions it has to perform to create the expected result.
- A powerful way to manage abstraction is through the use of **hierarchical classifications**.

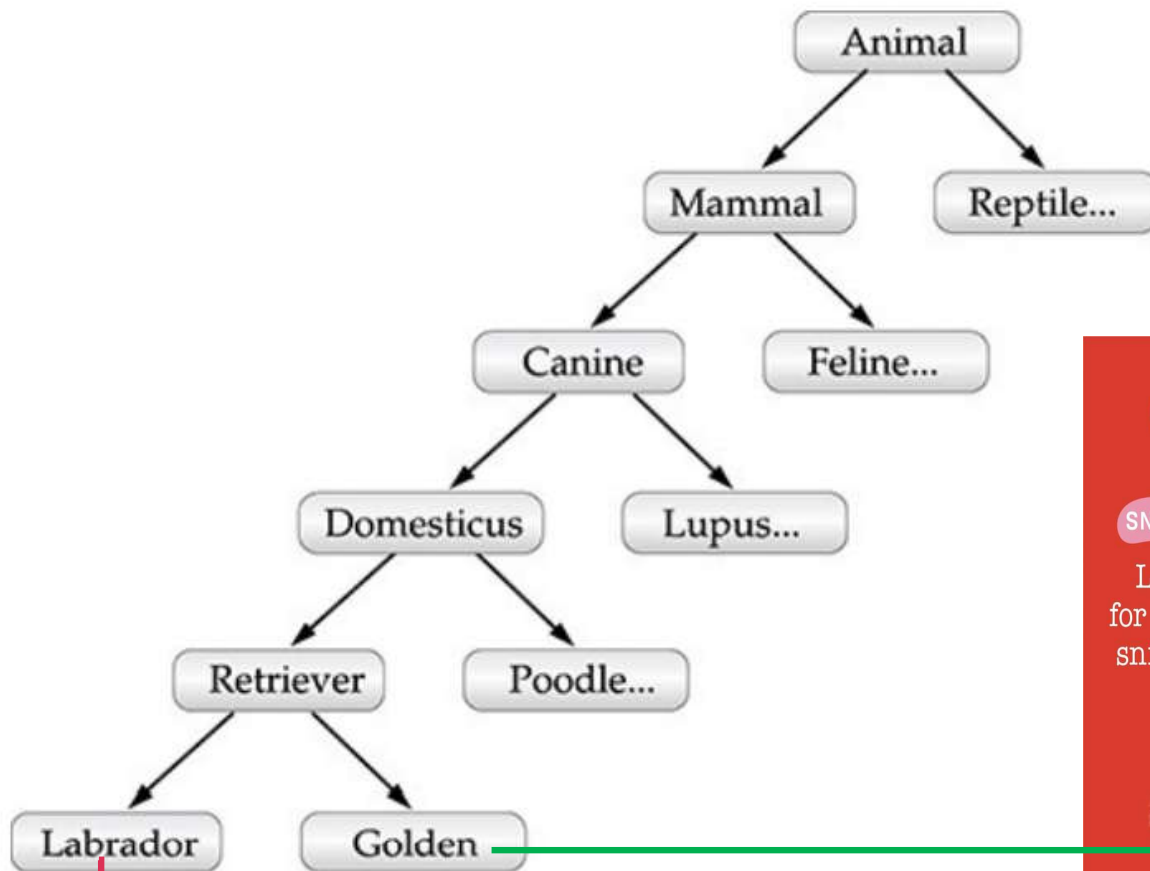
Encapsulation

- Is the **mechanism** that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.
- Ensures that **sensitive data** is **hidden** from users.
- To achieve this, you must:
 - declare variables/attributes as **private**
 - provide public **get** and **set** methods to access and update the value of a **private** variable



Inheritance

- In Java, it is possible to **inherit** (acquire) attributes and methods from one class to another, and creates **hierarchical classification**
 - **subclass** (child) - the class that **inherits** from another class.
 - **superclass** (parent) - the class **being inherited** from.
- **Inheritance** interacts with **encapsulation** as well.
 - If a given class encapsulates some attributes, then any subclass will have the same attributes **plus** any that it adds as part of its specialization.



GOLDEN RETRIEVERS

SNOUTS

Long
for extra
sniffing

COATS

Feathery
and soft

COLORS

Beige,
Honey,
Red



LABRADOR RETRIEVERS

SNOUTS

Wide for
extra
sniffing

COATS

Sleek
and soft

COLORS

Beige,
Brown,
Black



Source: Dailypaw

Polymorphism

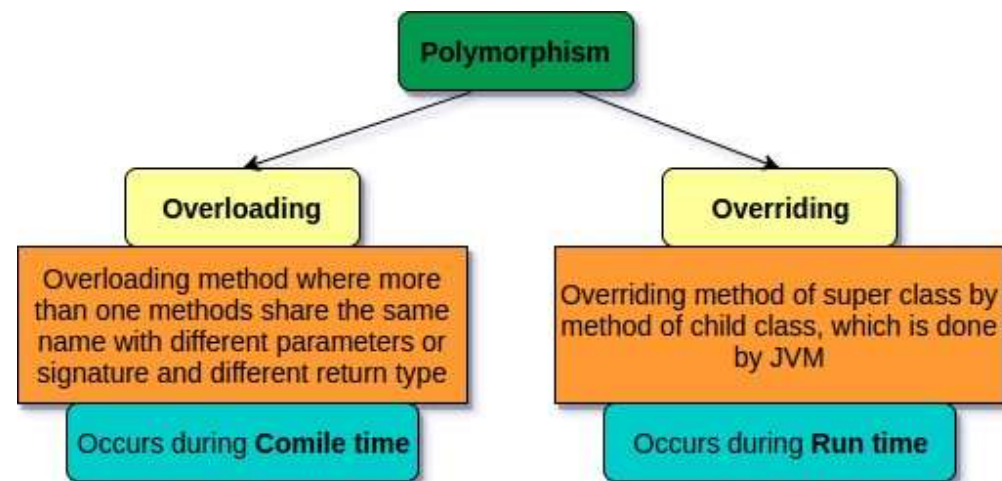
- Is often expressed by the phrase “one interface, multiple methods.”
- Is possible to design a generic interface to a group of related activities.
- Helps reduce complexity by allowing the same interface to be used to specify a general class of action.

```
class Animal {  
    public void animalSound() {  
        System.out.println("The animal makes a sound");  
    }  
}  
  
class Pig extends Animal {  
    public void animalSound() {  
        System.out.println("The pig says: wee wee");  
    }  
}  
  
class Dog extends Animal {  
    public void animalSound() {  
        System.out.println("The dog says: bow wow");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Animal myAnimal = new Animal(); // Create a Animal object  
        Animal myPig = new Pig(); // Create a Pig object  
        Animal myDog = new Dog(); // Create a Dog object  
        myAnimal.animalSound();  
        myPig.animalSound();  
        myDog.animalSound();  
    }  
}
```

Lecturer: Ngoc Tran, Ph.D.

Polymorphism

- E.g., a program requires three types of stacks. One stack is used for **integer** values, one for **floating-point** values, and one for **characters**.
- The algorithm that implements each stack is the same, even though the data being stored differs.
 - In a **non-object-oriented language**, three different sets of stack routines are created with each set using different names.
 - **Java** specifies a general set of stack routines that all share the same names.



References

1. Herbert Schildt, “Java - The Complete Reference”, 11th edition, Oracle Press, 2019. ISBN: 978-1-26-044024-9.
2. Kathy Sierra and Bert Bates, “Head First Java”, 2nd Edition, O’reilly Media Publisher, 2005. ISBN: 0596009208.
3. Oracle Documents, 2021.
4. Y. Daniel Liang , Lectures, “Introduction to Java Programming”.
5. Java tutorials, <https://www.w3schools.com/java/>.
6. Some of examples are copied from the Internet.